SOLID Principles

Object-Oriented Design Fundamentals with Java Examples

A practical guide to the five SOLID principles, each illustrated with a real Java design pattern, a violation example, and a corrected implementation.

Introduction

SOLID is an acronym for five design principles introduced by Robert C. Martin ("Uncle Bob") that help developers write software that is easier to maintain, extend, and test. Each letter stands for a principle that addresses a specific aspect of class and module design:

- S — Single Responsibility Principle (SRP)
- O — Open/Closed Principle (OCP)
- L — Liskov Substitution Principle (LSP)
- I — Interface Segregation Principle (ISP)
- D — Dependency Inversion Principle (DIP)

The sections below explain each principle, show a common violation in Java, present a corrected design, and connect the fix to a well-known design pattern (Strategy, Decorator, Template Method, Adapter, and Factory/Dependency Injection) so the ideas can be recognized in real codebases.

1. Single Responsibility Principle (SRP)

"A class should have only one reason to change." Each class should own exactly one piece of business logic or behavior, not several unrelated ones.

Violation

The Invoice class below mixes business logic, persistence, and presentation in one place. A change to the database layer or the print format forces a change to this class as well.

```
public class Invoice {

    private String customer;
    private double amount;

    public double calculateTotal() {
        // business logic
        return amount * 1.08;
    }

    // Responsibility #2: persistence
    public void saveToDatabase() {
        // JDBC code to insert invoice into DB
    }

    // Responsibility #3: presentation
    public void printInvoice() {
        System.out.println("Invoice for " + customer);
    }
}
```

Applying SRP

Splitting the responsibilities into three focused classes means each one has a single reason to change: pricing rules, storage, and formatting evolve independently.

```
public class Invoice {
```

```
    private String customer;
    private double amount;

    public double calculateTotal() {
        return amount * 1.08;
    }
}

public class InvoiceRepository {
    public void save(Invoice invoice) {
        // JDBC code to insert invoice into DB
    }
}

public class InvoicePrinter {
    public void print(Invoice invoice) {
        System.out.println("Invoice for " + invoice);
    }
}
```

Design pattern connection:

This separation is the foundation of the MVC (Model-View-Controller) pattern and of the Repository pattern — each layer (model, persistence, presentation) is isolated behind its own class, exactly as SRP requires.

2. Open/Closed Principle (OCP)

“Software entities should be open for extension, but closed for modification.” New behavior should be added by adding new code, not by editing tested, existing code.

Violation

The if/else chain below must be edited every time a new customer tier is introduced, risking regressions in already-working branches.

```
public class DiscountCalculator {

    public double apply(String customerType, double amount) {
        if (customerType.equals("REGULAR")) {
            return amount;
        } else if (customerType.equals("SILVER")) {
            return amount * 0.95;
        } else if (customerType.equals("GOLD")) {
            return amount * 0.90;
        }
        // Adding a new tier means editing this method again
        return amount;
    }
}
```

Applying OCP — the Strategy pattern

Each discount rule becomes its own class implementing a common interface. DiscountCalculator depends only on the interface, so new tiers are added without touching existing code.

```
public interface DiscountStrategy {
```

```
    double apply(double amount);
}

public class RegularDiscount implements DiscountStrategy {
    public double apply(double amount) { return amount; }
}

public class SilverDiscount implements DiscountStrategy {
    public double apply(double amount) { return amount * 0.95; }
}

public class GoldDiscount implements DiscountStrategy {
    public double apply(double amount) { return amount * 0.90; }
}

public class DiscountCalculator {
    private final DiscountStrategy strategy;

    public DiscountCalculator(DiscountStrategy strategy) {
        this.strategy = strategy;
    }

    public double apply(double amount) {
        return strategy.apply(amount);   // no branching needed
    }
}

// Adding a new tier only means adding a new class:
public class PlatinumDiscount implements DiscountStrategy {
    public double apply(double amount) { return amount * 0.85; }
}
```

Design pattern connection:

This is the Strategy pattern. It is the textbook way to satisfy OCP: encapsulate each interchangeable algorithm behind a common interface so the client class stays closed for modification while the set of strategies stays open for extension. The Decorator pattern achieves the same goal for adding responsibilities to objects at runtime without altering their class.

3. Liskov Substitution Principle (LSP)

“Objects of a superclass should be replaceable with objects of a subclass without breaking the application.” A subtype must honor the behavioral contract of its supertype, not just its method signatures.

Violation

Square extends Rectangle to reuse code, but overriding setWidth and setHeight to keep both sides equal violates the contract that setting width alone leaves height unchanged. Code written against Rectangle breaks when handed a Square.

```
public class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int w) { this.width = w; }
    public void setHeight(int h) { this.height = h; }
    public int getArea() { return width * height; }
}
```

```
// Square "is-a" Rectangle mathematically, but breaks behavior:
public class Square extends Rectangle {
    @Override
    public void setWidth(int w) {
        this.width = w;
        this.height = w;   // silently changes height too
    }

    @Override
    public void setHeight(int h) {
        this.width = h;
        this.height = h;   // silently changes width too
    }
}

// Client code that relies on Rectangle's contract breaks for Square:
Rectangle r = new Square();
r.setWidth(5);
r.setHeight(10);
assert r.getArea() == 50;   // fails: actual area is 100
```

Applying LSP

Instead of forcing an inheritance relationship that does not hold behaviorally, both shapes implement a common Shape abstraction and expose only the operations they can honor. Mutable setters that create the contract mismatch are removed entirely.

```
public interface Shape {
    int getArea();
}

public class Rectangle implements Shape {
    private final int width;
    private final int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public int getArea() { return width * height; }
}

public class Square implements Shape {
    private final int side;

    public Square(int side) { this.side = side; }

    public int getArea() { return side * side; }
}

// Any Shape can be substituted safely; neither subtype
// promises behavior it cannot honor.
List<Shape> shapes = List.of(new Rectangle(5, 10), new Square(4));
int total = shapes.stream().mapToInt(Shape::getArea).sum();
```

Design pattern connection:

This mirrors the Template Method pattern's discipline: a base type should define a contract that every subtype can fulfill without special-casing. LSP is also the guiding rule behind the Composite pattern, where leaf and composite nodes must be interchangeable through the same component interface.

4. Interface Segregation Principle (ISP)

“Clients should not be forced to depend on methods they do not use.” Prefer several small, role-specific interfaces over one large, general-purpose interface.

Violation

The fat Worker interface forces every implementer to provide eat() and sleep(), even classes such as RobotWorker for which those operations are meaningless.

```
public interface Worker {
    void work();
    void eat();
    void sleep();
}

// A robot is forced to implement methods that make no sense for it:
public class RobotWorker implements Worker {
    public void work() { /* ... */ }

    public void eat() {
        throw new UnsupportedOperationException("Robots don't eat");
    }

    public void sleep() {
        throw new UnsupportedOperationException("Robots don't sleep");
    }
}
```

Applying ISP

Splitting Worker into Workable, Feedable, and Sleepable lets each class implement only the roles that apply to it.

```
public interface Workable {
    void work();
}

public interface Feedable {
    void eat();
}

public interface Sleepable {
    void sleep();
}

public class HumanWorker implements Workable, Feedable, Sleepable {
    public void work()  { /* ... */ }
    public void eat()   { /* ... */ }
    public void sleep() { /* ... */ }
}
```

```
// RobotWorker only implements the interface it can genuinely support:
public class RobotWorker implements Workable {
    public void work() { /* ... */ }
}
```

Design pattern connection:

This is the same discipline behind the Adapter pattern, which exposes a narrow, client-specific interface over a larger or incompatible one instead of requiring clients to depend on the whole surface area. Many Java core interfaces (Runnable, Comparable, Comparator) are deliberately small for the same reason.

5. Dependency Inversion Principle (DIP)

“High-level modules should not depend on low-level modules; both should depend on abstractions.” Depend on interfaces, not concrete implementations, and let a class's dependencies be supplied from the outside.

Violation

OrderService, a high-level module, is hard-wired to MySqlDatabase, a low-level implementation detail. Changing databases requires modifying OrderService directly.

```
public class MySqlDatabase {
    public void save(String data) {
        // MySQL-specific save logic
    }
}

public class OrderService {
    // High-level module depends directly on a low-level, concrete class
    private final MySqlDatabase database = new MySqlDatabase();

    public void placeOrder(String order) {
        database.save(order);
    }
}
// Switching databases means editing OrderService itself.
```

Applying DIP

Both OrderService and MySqlDatabase now depend on the Database abstraction. The concrete implementation is constructed elsewhere and injected into OrderService, decoupling policy from detail.

```
public interface Database {
    void save(String data);
}

public class MySqlDatabase implements Database {
    public void save(String data) { /* MySQL-specific save logic */ }
}

public class MongoDatabase implements Database {
    public void save(String data) { /* MongoDB-specific save logic */ }
}

public class OrderService {
```

```
    private final Database database;   // depends on an abstraction

    // Dependency is injected, not constructed internally
    public OrderService(Database database) {
        this.database = database;
    }

    public void placeOrder(String order) {
        database.save(order);
    }
}

// Composition happens outside OrderService, e.g. via a factory:
public class DatabaseFactory {
    public static Database create(String type) {
        return switch (type) {
            case "mongo" -> new MongoDatabase();
            default -> new MySqlDatabase();
        };
    }
}

OrderService service = new OrderService(DatabaseFactory.create("mongo"));
```

Design pattern connection:

This is Dependency Injection, closely paired with the Factory pattern (and Abstract Factory for families of related objects) which is responsible for constructing the concrete implementation so the high-level class never needs to know about it. Frameworks such as Spring build their entire IoC container around this principle.

Summary

The table below recaps each principle alongside the Java design pattern most commonly used to satisfy it.

Letter	Principle	Core Idea	Related Pattern
S	Single Responsibility	One reason to change per class	MVC / Repository
O	Open/Closed	Extend behavior without editing existing code	Strategy / Decorator
L	Liskov Substitution	Subtypes must honor the base contract	Template Method / Composite
I	Interface Segregation	Prefer small, role-specific interfaces	Adapter
D	Dependency Inversion	Depend on abstractions, inject implementations	Factory / Dependency Injection

Applied together, the SOLID principles push toward designs made of small, focused, loosely coupled classes composed behind abstractions — the same qualities that make the classic Gang-of-Four design patterns effective in real Java codebases.